

Name : UDHAYARAJAN J

Class : MCA - I

Subject : Datastructure using Java

Rollno : 2202561

Type : CSE-4 (Case Study on LinkedList)

Date : 2026-04-10

- Q. Read the real world problem and describe which linked list is used to implement solution. Use below format to describe the answer.

Requirement Analysis.

Best choice.

Why.

Structure design.

Conditions.

problem statements:

- ① Music playlist (without looping).

Requirement Analysis:

In this scenario songs are played one after another in sequence. The user should be able to move to the next song and also return to the previous song if needed. However, there is no requirement to automatically loop back from the last song to the first song.

Best choice:

The most suitable data structure for this requirement is a Doubly Linked List.

Why:

Supports both forward (next) and backward (previous)

Navigation

No need for circular linking, since loop is not required.

Structure Design:

class Node {
 String song;
 Node next;
}

Node First;

Node Last;

Node (Song) {

}

§
§

Condition: (linked list) singly, doubly

prev == Null // first song

next == Null // last song

or Even we stored first and last of linking

② Undo/Redo system (like Microsoft Word)

Requirements analysis:

An undo/redo system needs to maintain a history of user actions. The user should be able to move backward to undo an action and move forward to redo an action.

This requires efficient navigation in both directions through previously performed operations.

Best choice:

The most suitable data structure is a Doubly Linked List.

Why:

Allow traversal in both directions.
Efficient movement between previous and next states.

Structure Design:

class Node {

Node prev;

Node next;

object actions;

Node(actions) {

.....

.....

}

}

Conditions

prev == null // no more undo

next == null // no more redo

When new action is added, all nodes after the current node must be deleted to maintain correct history

③ Game Turns

Requirements Analysis:

In a game with multiple players, each player gets a turn in a fixed order after the last player finishes their turn the game should continue again from the first player. This cycle repeats continuously throughout the game.

Best choice:

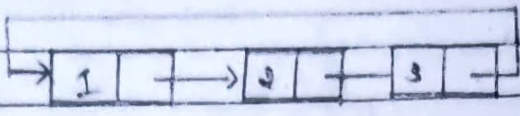
The most suitable data structure is a circular linked list.

Why:

A circular linked list allows continuous traversal without reaching a null end: once the last player completes their turn, the structure automatically moves to the first player, ensuring smooth repeated turn rotation.

Structure Design:

```
class Node {
    Node prev;
    Node next;
    Node(player) {
```



Each node points to the next player. The last node points back to the head forming a circular structure.

Condition:

There is no Null pointer in the list. The condition $\text{last.next} = \text{head}$ must always be maintained to ensure continuous looping of turns.

② Book reading List

Requirement Analysis:

In a book reading list, books are arranged in a sequence for reading. The user may move forward to the next book and may also want to go back to a previously read book. There is no requirement for circular traversal.

Best choice:

The most suitable data structure is a Doubly Linked List.

Why:

A doubly linked list allows traversal in both forward and backward directions, making it convenient for users to navigate between books. It efficiently supports revisiting previously read books as well as continuing to the next one.

Structure design:

class Node {

Node prev;

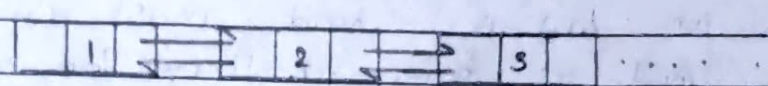
Node next;

Node(Node n) {

.....
.....

}

}



Conditions:

The first node has prev = null and last node has next = null. There is no circular connection in the list.

⑤ Round-Robin CPU Scheduling.

Requirement Analysis:

In round-robin CPU scheduling, multiple processes are executed in cyclic order. Each process is given a fixed time slice (quantum) and after its execution the CPU moves to the next process. Once the last process is executed, the scheduling continues again from the first process.

Best choice:

The most suitable data structure is a circular linked list.

Why:

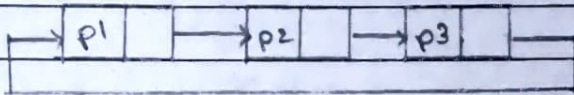
A circular linked list naturally supports cyclic traversal. After reaching the last process, it automatically continues with the first process.

Date _____
Page _____

with requiring any reset, making it ideal for round-robin scheduling.

Structure design:

```
class Node {  
    Node next;  
    Node(process) {  
        .....  
    }  
}
```



Each node points to the next process.

Conditions:

There is no NULL pointer in the list. The condition $\text{last.next} = \text{head}$ must always be maintained to ensure continuous execution of process in a loop.